Dynamic Universal Accumulators from Lattices

Madusha Chathurangi[1], Qinyi Li[1], and Xavier Boyen[2]

[1] Griffith University, Brisbane, Australia
qinyi.li@griffith.edu.au
[2] Queensland University of Technology, Brisbane, Australia

Abstract. Dynamic universal accumulators (DUAs) are cryptographic primitives that compress a set of elements into a short representation while supporting dynamic updates, as well as efficient proofs of membership and non-membership. These systems are useful in several practical applications, such as revocation schemes and privacy-preserving authentication mechanisms. However, existing "tree-based" lattice accumulators usually provide only one of the two key properties, either dynamicity or universality. Moreover, prior Merkle-tree-based accumulators require logarithmic-size updates and rely on Stern-type zero-knowledge proofs, resulting in large proofs and a relatively high soundness error. To overcome these limitations, we propose the first lattice-based DUA. Based on Verkle trees, our construction accumulates directly from the leaves via a linear function, enabling more efficient, *stateless* updates. By differentiating the ranges of the short secret values, our construction allows very efficient membership and non-membership zero-knowledge proofs. Moreover, compared to the existing accumulators, its witness does not require sibling information, thereby reducing communication overhead. These features make the proposed accumulator well-suited for membership revocation in post-quantum, privacy-preserving applications.

Keywords: Dynamic · Universal · Accumulators · Post-Quantum

1 Introduction

Efficiently managing dynamic data sets is a core challenge in many cryptographic applications, especially when systems must support verifiable set membership (or non-membership) without incurring high storage or communication costs. These sets, such as revocation lists, records of spent tokens, or registries of authorized participants, must be maintained, updated, and verified as they grow over time. However, storing, updating, and distributing these sets in their entirety can impose significant computational and communication overhead, particularly as their size increases. Cryptographic accumulators address this challenge by providing a succinct representation of a set that allows any party to generate compact proofs of membership (or non-membership) without revealing the full list of elements. First introduced by Benaloh and de Mare [3], accumulators have become a fundamental primitive in the design of privacy-preserving and efficient cryptographic protocols, including group signatures [13], ring signatures [13],

anonymous credentials [10], identity escrow schemes [5], electronic cash systems such as Zerocoin [20], and membership testing [3].

For accumulators to be practical in such applications, two properties are particularly important: dynamicity and universality. An accumulator that supports both is referred to as a dynamic universal accumulator (DUA), where "D" denotes dynamic and "U" denotes universal. Dynamicity [5] means that the accumulator be efficiently updatable when elements are added to or removed from the set. Hence, constructions that require reaccumulating all values or reconstructing the entire tree after each update are inefficient and do not satisfy the intended notion of dynamicity. Universality [12] means that the accumulator supports both membership and non-membership proofs. These two properties are particularly important in practical settings where the underlying set changes over time and both types of proofs may be required. For example, consider a secure facility that maintains a set of authorized persons. Since this set may change over time as people are granted or denied access, efficient updates are essential. At the same time, an authorized person may need to prove membership in the set to enter the facility, while it may also be necessary to prove that an unauthorized person is not in the set.

1.1 Motivations

Although lattice-based accumulators achieving dynamicity [10] or universality [26] have been studied, no existing post-quantum construction achieves both properties simultaneously. In particular, there is still no post-quantum *tree-based* DUA, despite the practical importance [20,3].

Tree-based constructions, especially Merkle-tree-based ones [15,16,26], are important because they offer a practical and conceptually simple framework for accumulators, rely on collision-resistant hash functions, and support authentication with logarithmic complexity. The current approach to tree-based post-quantum lattice accumulator constructions is primarily based on the Merkle-tree paradigm. Prior works introduced a lattice-based Merkle-tree static accumulator [13] and later extended it to dynamic [16] and universal variants [26]. However, the universal accumulator of Yu et al. [26] requires the accumulated secrets to be ordered, making dynamicity difficult to achieve in practice, since insertions or deletions may affect the ordering of other elements, require reconstructing the tree, and thereby prevent efficient updates. Hence, to the best of our knowledge, no existing "tree-based" construction provides both dynamicity and universality, unless two trees are maintained, which incurs higher overhead [26].

Moreover, in binary-tree-based accumulators [15,16,26], the number of zero-knowledge relations increases linearly with the tree depth, $\log_2(N)$. While this dependence is logarithmic in the number of leaves, it remains significant because deeper trees directly lead to larger proof overhead. For instance, with 2^{16} leaves, a binary tree has depth 16, whereas a non-binary tree with branching factor 4 has depth 8, thereby halving the number of levels and the corresponding number of proof relations. This led us to consider a higher branching factor.

Finally, existing "tree-based" lattice accumulators do not support stateless updates and direct updates to the root. The term *stateless updatability* means that updates require only the position where the message vector changed, along with the old and new entries, or even just their difference, without requiring the update store or recomputing the full set. Existing binary-tree-based lattice accumulators [16] do not achieve this goal; in particular, their witnesses contain sibling information, since authentication depends not only on the nodes along a root-to-leaf path, but also on the sibling hashes associated with that path. Consequently, insertions, deletions, and modifications require maintaining sibling nodes [16], which is undesirable in dynamic settings with frequent updates.

These limitations motivated us to explore an alternative approach based on vector commitments, aiming to construct a DUA that overcomes the limitations of existing Merkle-tree-based accumulators.

To the best of our knowledge, Zhao et al. [27] is the only work that attempts to build a lattice DUA from the vector commitment of Peikert et al. [21]. However, this construction cannot yet be considered as a fully realized DUA. In particular, in Zhao et al.'s construction [27], elements are represented in an index-based way: members are given all-one vectors, while non-members are given all-zero vectors. Because of this, membership is linked only to a position in the accumulator, not to a unique user value such as a public key. This is insufficient for applications where each user should have their own secret key and corresponding public key. This is especially important in group signatures, where each valid signature should open to one specific user. In addition, because the construction is linear (non-tree-based), the key size grows linearly with the number of members, which becomes inefficient as the number of users increases. These limitations motivate us to construct a tree-based accumulator from vector commitments that accumulates unique user keys and achieves better scalability.

1.2 Our Contributions and Approaches

The First Efficient Dynamic Universal Lattice Accumulator. We propose a quantum-secure lattice-based DUA constructed as a Verkle tree over module lattices. The design is inspired by the vector commitment framework of Peikert et al. [21] and adopts an b-ary hierarchical structure to organize commitments efficiently. Our construction uses lattice-based vector commitments at each internal node, enabling compact binding to a dynamic set of elements together with efficient position-specific openings and update operations. The accumulator is constructed over a set $\mathcal{X} = \{\mathbf{x}_0, \ldots, \mathbf{x}_{N-1}\}$, with the maximum number of users fixed to $N = b^h$. The construction follows an SIS-based Verkle tree paradigm. Let $\mathbf{G}^{-1}()$ represent the gadget matrix which does component-wise binary decomposition to its inputs and $\mathbf{B}$ be a public matrix. Each user holds a secret binary vector $\mathbf{x}_j$ which derives $\mathbf{m}_j = \mathbf{G}^{-1}(\mathbf{y}_j)$, where $\mathbf{y}_j = \mathbf{B}\mathbf{x}_j$. The bottom layer of the tree consists of the elements $\mathbf{m}_j$ themselves. The elements of $\mathcal{X}$ are committed into the tree, and the resulting root value $\mathbf{c}$ is published as the accumulator value.

Compared to traditional Merkle hash trees, our design provides several advantages. While prior schemes employ binary trees, yielding a height of $\log_2(N)$, our construction adopts any branching factor $b \geq 2$. When $b = 2$, our construction follows a binary-tree structure. When $b > 2$, we can reduce the tree height to $h = \log_b(N)$, which, in turn, lowers the cost of witness generation and verification. This is particularly useful when using the accumulator attached zero-knowledge proof (ZKP) system to prove the knowledge of membership, as the proof complexity is linear in h and amortisation is usually available in each depth across all b nodes.

Cryptographic accumulators are useful when a compatible ZKP system is available to efficiently prove the knowledge of membership and non-membership, in zero-knowledge ways. We devise a simple lattice-based ZKP component efficiently proving membership and non-membership. We do this by structuring users' secrets: let $\mathbf{e}$ be a constant, coordinate-equal vector with coordinates greater than 2; members prove their public keys correspond to binary secrets $\mathbf{x}_j$ and non-members prove their public keys have secrets that are $\mathbf{e}$-shifted binary vectors, i.e., $\mathbf{x}_j + \mathbf{e}$. We demonstrate that this design is fully compatible with the state-of-the-art ZKP system [17], such that the above-mentioned knowledge can be practically proved in zero knowledge.

Last but not least, compared to Merkle-tree membership proofs, witness size is $\log_b(N)$, which is significantly less than $\log_2(N)$. Unlike Merkle-tree-based constructions, which require sibling hashes along the authentication path, our scheme removes this dependency by using a *sibling-free witness*, thereby significantly reducing communication overhead. Moreover, we achieve *stateless updates*, meaning that an update requires only the modified position together with the old and new values. We summarise a comparison among existing lattice-based accumulators in Table 1, showing that our construction is the first to achieve dynamicity, universality, and supporting stateless accumulation updates.

Table 1. Comparison of "tree-based" quantumly secure Lattice Accumulators.

Scheme	Model	Struct.	|PK|	|Witness|	|Update|	Upd. Type
Libert et al. [13,15]	Static	Merkle	$\mathcal{O}(nm \log q)$	$\mathcal{O}(\log_2 N)$	✗	✗
Ling et al. [16]	Dynamic	Merkle	$\mathcal{O}(nm \log q)$	$\mathcal{O}(\log_2 N)$	$\mathcal{O}(\log_2 N)$	Stateful
Yu et al. [26]	Universal	Merkle	$\mathcal{O}(nm \log q)$	$\mathcal{O}(\log_2 N)$	✗	✗
Ours	Dyn. Univ.	Verkle	$\mathcal{O}(nwb \log q)$	$\mathcal{O}(\log_b N)$	$\mathcal{O}(1)$	Stateless

Here, $N = b^h$ denotes the total number of leaves, where b is the branching factor and h is the tree height. The term *static* refers to an accumulator in which the set is fixed after setup, *dynamic* refers to an accumulator that supports adding and removing elements after setup, and *universal* refers to an accumulator that supports both membership and non-membership proofs.

As shown in Table 1, although proposed DUA is tree-based, the root value can be updated directly through a linear function without recomputing other nodes. As a result, it achieves update complexity $\mathcal{O}(1)$, in contrast to the logarithmic

update complexity of Ling et al.'s scheme [16]. Furthermore, our construction benefits more significantly from a higher branching factor than Merkle-tree-based schemes. While increasing the branching factor also reduces the height of a Merkle tree, the prover must still reveal the other $b-1$ sibling nodes at each level. By contrast, our construction does not require sibling disclosure, so reducing the tree height from $\log_2 N$ to $\log_b N$ directly results in lower witness overhead and a shorter proof path.

Applications to Membership Revocation. DUAs can be useful in many real-life settings. For example, consider a group of people with a medical condition that makes them eligible for discounted medicine but restricts them from entering certain facilities, such as swimming pools [**?**]. In this case, these individuals should be able to efficiently prove membership in the relevant set at a pharmacy, while others should be able to prove non-membership when access to restricted areas is checked. Generally, these applications require a mechanism that supports both membership and non-membership proofs while allowing the underlying set to change over time.

Among such applications, the proposed DUA is well-suited to revocation-related applications such as certificate revocation and anonymous credential systems [8]. In particular, in Revocable Anonymous Credential (RAC) systems, a user must prove not only possession of a valid credential but also that it has not been revoked, while preserving anonymity. Our DUA is suitable for this setting because it represents a large set of credential identifiers, such as serial numbers, by a single short accumulator value, avoiding the need to distribute long revocation lists. Its dynamic property enables the revocation authority to efficiently update the accumulated set as credentials are revoked or renewed, while its universal property supports either membership or non-membership proofs, depending on the revocation design. For example, our approach can be used to transform the existing lattice-based anonymous credential scheme of Jeudy et al. [8] into an RAC. Furthermore, in addition to revocation mechanisms, the proposed DUA may also be applicable to other dynamic membership settings, including lattice-based group signatures [13], authentication mechanisms, attribute-based access control, authenticated dictionaries, redactable signatures, and consortium blockchain systems.

Moreover, many applications of DUAs, such as anonymous credentials [8], group signatures [13], and revocation mechanisms, require the use of zero knowledge proofs. However, existing lattice-based accumulators [13,16,26] often rely on Stern-type protocols [23], which can lead to large proof sizes and relatively high soundness errors. In contrast, our framework enables efficient proofs of membership or non-membership in zero knowledge, compatible with Lyubashevsky-style proof systems [18]. In fact, membership of an element $\mathbf{x} \in \mathcal{X}$ is certified by a witness wit_x relative to the accumulator root $\mathbf{c}$ and proven via a zero-knowledge argument of knowledge, following Lyubashevsky [17]. Consequently, the proposed DUA is also well-suited for lattice-based privacy-preserving revocation mechanisms that require efficient zero-knowledge proofs.

1.3 Related Work

A wide range of cryptographic accumulators have been proposed over the past decades, primarily based on classical cryptographic primitives such as RSA variants [20], and bilinear pairings [5,4]. However, these rely on number-theoretic assumptions: discrete logarithms and integer factorization, both of which are vulnerable to Shor's quantum algorithm [22]. As a result, many of these widely used accumulator schemes are no longer considered secure against quantum adversaries, motivating the construction of post-quantum cryptographic alternatives: lattice-based cryptography. The first lattice-based accumulator was introduced in 2015 based on aggregate signatures [9]. Later, several lattice-based accumulators were proposed that typically rely on Merkle hash trees to encode sets and support membership proofs. In 2016, Libert et al. [13] introduced a Merkle tree-based static accumulator, later extended to its dynamic [16] and universal [26] variants. These designs [13,16,26] generally use Stern-type zero-knowledge proofs [23], which suffer from high soundness error (2/3), requiring $\kappa = \omega(\log \lambda)$ repetitions to achieve negligible error [14,26]. In contrast, as summarized in Table 1, our accumulator is fully dynamic and universal, built on module lattices using a Verkle tree; hence, we introduce an efficient lattice-based DUA suitable for frequent updates.

2 Preliminaries

Notations. For positive integer n, define $[n] := \{1, 2, \ldots, n\}$, and for any integers $a \leq b$, define $[a, b] := \{a, a+1, \ldots, b\}$. We denote by $x \leftarrow S$ sampling x uniformly at random from a finite set S, and $x \leftarrow \mathcal{D}$ to denote that x is sampled according to a distribution $\mathcal{D}$. We use $\otimes$ to denote the vector cross product.

Let q be an odd prime. For a power-of-two integer n, let $\mathcal{R} = \mathbb{Z}[X]/(X^n + 1)$ and $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^n + 1)$, where $X^n + 1$ is the $2n$-th cyclotomic polynomial. Unless stated otherwise, lower-case letters denote elements of $\mathcal{R}$ or $\mathcal{R}_q$, bold lower-case letters denote column vectors with coordinates in $\mathcal{R}$ or $\mathcal{R}_q$, and bold upper-case letters denote matrices over $\mathcal{R}$ or $\mathcal{R}_q$. Let $\theta : \mathcal{R} \to \mathbb{Z}^n$ denote the coefficient embedding, $\theta\left(\sum_{i=0}^{n-1} r_i X^i\right) = (r_0, \ldots, r_{n-1})^\mathsf{T}$. Define $S_{\mathsf{bin}} = \theta^{-1}(\{0,1\}^n)$ and $S_1 = \theta^{-1}(\{-1, 0, 1\}^n)$. For $p \geq 1$, set $\|r\|_p := \|\theta(r)\|_p$, and denote by $D_{R,\sigma}$ the discrete Gaussian distribution $\theta^{-1}(D_{\theta(R),\sigma})$ over $\mathcal{R}$.

For $w \in \mathbb{Z}_q$, we define $\|w\|_\infty := |w \bmod^{\pm} q|$. For a polynomial $w = w_0 + w_1 X + \cdots + w_{n-1} X^{n-1} \in \mathcal{R}$, the ℓ_∞ norm is defined as $\|w\|_\infty = \max_j \|w_j\|_\infty$. For a vector $\mathbf{w} = (w_1, \ldots, w_k) \in \mathcal{R}^k$, we define $\|\mathbf{w}\|_\infty = \max_j \|w_j\|_\infty$. We define the Euclidean norm of $\mathbf{w}$ by $\|\mathbf{w}\| = ||\sqrt{\sum_{i=0}^{n-1} w_i^2}||$.

2.1 Short Integer Solution [11,18]

We consider the lattice assumptions underlying our construction, specifically the Module-SIS (MSIS) problem, which are instantiated over $\mathcal{R}_q$.

Definition 1 (Module-SIS). *Given* $\mathbf{A} \leftarrow \mathcal{R}_q^{d\times m}$*, we say the Module-SIS problem* $\mathrm{MSIS}_{d,m,B}$ *with parameters* $d, m > 0$ *and* $0 < B < q$ *is hard if the following probability is negligible in* n*:* $\Pr\left[\mathbf{Az} = \mathbf{0} \wedge \|\mathbf{z}\|_\infty \le B \mid \mathbf{z} \leftarrow \mathcal{A}(\mathbf{A})\right]$.

2.2 Lattice Trapdoor and Preimage Sampling

We use a public gadget matrix $\mathbf{G} \in \mathcal{R}_q^{d\times w}$ as in the gadget–trapdoor framework. Let $w = d\lceil \log q \rceil$ and define $\mathbf{g}^\top = (1, 2, 4, \ldots, 2^{\lceil \log q \rceil - 1})$. The gadget matrix is

$$\mathbf{G} = \mathbf{I}_d \otimes \mathbf{g}^\top = \begin{bmatrix} \mathbf{g}^\top & & & \\ & \mathbf{g}^\top & & \\ & & \ddots & \\ & & & \mathbf{g}^\top \end{bmatrix} \in \mathcal{R}_q^{d\times w}$$

The matrix $\mathbf{G}$ has a deterministic inverse $\mathbf{G}^{-1} : \mathcal{R}_q^d \to \mathcal{R}^w$ such that $\mathbf{G}\mathbf{G}^{-1}(\mathbf{u}) = \mathbf{u}$ and $\|\mathbf{G}^{-1}(\mathbf{u})\| \le \sqrt{w}$ for all $\mathbf{u} \in \mathcal{R}_q^d$.

Theorem 1 ([7,19]). *There exist probabilistic polynomial-time algorithms* $\mathsf{TrapGen}$*,* $\mathsf{SampleD}$*, and* $\mathsf{SamplePre}$*, together with a fixed gadget matrix* $\mathbf{G} \in \mathcal{R}_q^{d\times w}$*, such that the following statements hold. Let* $m = \bar{m} + w$*.*

1. *Given as input a matrix* $\bar{\mathbf{A}} \in \mathcal{R}_q^{d\times \bar{m}}$ *and a matrix* $\mathbf{H}^* \in \mathcal{R}_q^{d\times d}$*, the algorithm* $\mathsf{TrapGen}(\bar{\mathbf{A}}, \mathbf{H}^*)$ *produces a pair* $(\mathbf{A}, \mathbf{T}) \in \mathcal{R}_q^{d\times m} \times \mathcal{R}^{\bar{m}\times m}$ *satisfying* $s_1(\mathbf{T}) \le s_T = O(\sqrt{m})$ *and* $\mathbf{A} = \bar{\mathbf{A}}\mathbf{T} + [\,\mathbf{0} \mid \mathbf{H}^*\mathbf{G}\,] \in \mathcal{R}_q^{d\times m}$*.*
2. *Assume* $\bar{m} \ge 2d \log q$ *and let* $\mathbf{H}^* \in \mathcal{R}_q^{d\times d}$ *be arbitrary. If* $\bar{\mathbf{A}} \leftarrow \mathcal{R}_q^{d\times \bar{m}}$ *is sampled uniformly at random and* $(\mathbf{A}, \mathbf{T}) \leftarrow \mathsf{TrapGen}(\bar{\mathbf{A}}, \mathbf{H}^*)$*, then the resulting matrix* $\mathbf{A}$ *is statistically close to uniform.*
3. *Let* $\mathbf{H}^*, \mathbf{H} \in \mathcal{R}_q^{d\times d}$ *satisfy that* $\mathbf{H}^* - \mathbf{H}$ *is invertible. For any* $\bar{\mathbf{A}} \in \mathcal{R}_q^{d\times \bar{m}}$*, any* $s \ge s_T \cdot \omega$*, and any positive integer* k*, define* $\mathbf{A}' = \mathbf{A} - [\,\mathbf{0} \mid \mathbf{HG}\,]$*. The following properties then hold:*
   (a) *If* $\mathbf{R} \leftarrow \mathsf{SampleD}(\mathcal{R}^{1\times k}, s)$*, then the matrix* $\mathbf{U} = \mathbf{A}'\mathbf{R} \in \mathcal{R}_q^{d\times k}$ *is distributed within statistical distance* $k \cdot \mathrm{negl}(n)$ *of uniform over* $\mathcal{R}_q^{n\times k}$*.*
   (b) *For any matrix* $\mathbf{U} \in \mathcal{R}_q^{d\times k}$*, the algorithm* $\mathsf{SamplePre}(\mathbf{T}, \mathbf{A}', \mathbf{U}, s)$ *outputs a matrix* $\mathbf{R} \in \mathcal{R}^{m\times k}$ *such that* $\|\mathbf{R}\| \le s\sqrt{m}$ *and* $s_1(\mathbf{R}) = O\big(s\sqrt{m+k}\big)$*, and the distribution of* $\mathbf{R}$ *is within* $k \cdot \mathrm{negl}(d)$ *statistical distance of the distribution* $\mathsf{SampleD}(\mathcal{R}^{1\times k}, s)$ *conditioned on the event* $\mathbf{A}'\mathbf{R} = \mathbf{U}$*.*

3 Syntax of Cryptographic Primitives

In this section, we present the formal syntax of DUA together with the relevant security notions. The definitions are expressed using notation that is standard in the accumulator literature [6,2].

3.1 Dynamic Universal Accumulator

Definition 2 (Dynamic Universal Accumulator). *A* dynamic universal accumulator *(DUA) consists of the following polynomial-time algorithms:*

1. $(\mathsf{sk}_{\mathsf{acc}}, \mathsf{pk}_{\mathsf{acc}}) \leftarrow \mathsf{Gen}(1^\lambda, t)$*: On input a security parameter* λ *and an upper bound* t *on the maximum number of accumulated elements, outputs a key pair* $(\mathsf{sk}_{\mathsf{acc}}, \mathsf{pk}_{\mathsf{acc}})$*, where* $\mathsf{sk}_{\mathsf{acc}} = \emptyset$ *if no trapdoor exists.*
2. $(\mathsf{acc}_{\mathcal{X}}, \mathsf{aux}) \leftarrow \mathsf{Eval}(\mathsf{pk}_{\mathsf{acc}}, \mathcal{X})$*: On input public key* $\mathsf{pk}_{\mathsf{acc}}$*, and a set* $\mathcal{X}$*, outputs an accumulator value* $\mathsf{acc}_{\mathcal{X}}$ *together with auxiliary information* aux*.*
3. $(\mathsf{acc}_{\mathcal{X}'}, \mathsf{aux}') \leftarrow \mathsf{Update}((\mathsf{sk}_{\mathsf{acc}}, \mathsf{pk}_{\mathsf{acc}}), \mathsf{acc}_{\mathcal{X}}, \mathsf{aux}, x, x')$*: If* $x \notin \mathcal{X}$*, outputs* $\perp$*. Otherwise, define* $\mathcal{X}' = \big(\mathcal{X} \setminus \{x\}\big) \cup \{x'\}$ *and output the updated accumulator* $\mathsf{acc}_{\mathcal{X}'}$ *and auxiliary information* aux'*.*
4. $\mathsf{wit}_x \leftarrow \mathsf{WitCreate}((\mathsf{sk}_{\mathsf{acc}}, \mathsf{pk}_{\mathsf{acc}}), \mathsf{acc}_{\mathcal{X}}, \mathsf{aux}, x, \mathsf{type})$*: Outputs a witness* wit_x *for element* x*, where* $\mathsf{type} = 0$ *denotes a membership witness* ($x \in \mathcal{X}$) *and* $\mathsf{type} = 1$ *denotes a non-membership witness* ($x \notin \mathcal{X}$)*. Outputs* $\perp$ *if the requested witness does not exist.*
5. $\{\mathsf{true}, \mathsf{false}\} \leftarrow \mathsf{Verify}(\mathsf{pk}_{\mathsf{acc}}, \mathsf{acc}_{\mathcal{X}}, \mathsf{wit}_x, x, \mathsf{type})$*: Outputs* true *if* wit_x *is a valid witness for* x *w.r.t* $\mathsf{acc}_{\mathcal{X}}$ *and the given* type*, and* false *otherwise.*

A DUA is *correct* if, for all $\lambda \in \mathbb{N}$ and all $t = \mathsf{poly}(\lambda)$, accumulator $(\mathsf{acc}_{\mathcal{X}}, \mathsf{aux})$ representing $\mathcal{X}$ for $\mathcal{X} \subseteq \mathcal{M}$, and for any element $x \in \mathcal{M}$ with an up-to-date witness wit_x, it holds that

$$\Pr[\mathsf{Verify}(\mathsf{pk}_{\mathsf{acc}}, \mathsf{acc}_{\mathcal{X}}, \mathsf{wit}_x, x, \mathsf{type}) = \mathsf{true}] \geq 1 - \mathsf{negl}(\lambda),$$

where $\mathsf{type} = 0$ if $x \in \mathcal{X}$ and $\mathsf{type} = 1$ otherwise.

Collision Resistance ensures that it is computationally infeasible to produce a valid membership witness for any element $x \notin \mathcal{X}$ (or a valid non-membership witness for any element $x \in \mathcal{X}$).

Definition 3 (Collision Resistance). *A dynamic universal accumulator for domain* $\mathcal{M}$ *is said to be* collision resistant *if for any probabilistic polynomial-time adversary* $\mathcal{A}$ *with oracle access to* $\mathcal{O}$*, there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that for all* $\lambda \in \mathbb{N}$*,*

$$\Pr\left[\begin{array}{l}(\mathsf{sk}_{\mathsf{acc}}, \mathsf{pk}_{\mathsf{acc}}) \leftarrow \mathsf{Gen}(1^\lambda, t);\\ (\mathsf{acc}_{\mathcal{X}}, \mathsf{aux}) \leftarrow \mathsf{Eval}(\mathsf{pk}_{\mathsf{acc}}, \mathcal{X});\\ (x^*, \mathsf{wit}^*, \mathsf{type}^*) \leftarrow \mathcal{A}^{\mathcal{O}}(\mathsf{pk}_{\mathsf{acc}}, \mathsf{acc}_{\mathcal{X}}) :\\ \mathsf{Verify}(\mathsf{pk}_{\mathsf{acc}}, \mathsf{acc}_{\mathcal{X}}, \mathsf{wit}^*, x^*, \mathsf{type}^*) = \mathsf{true} \wedge\\ \big((\mathsf{type}^* = 0 \wedge x^* \notin \mathcal{X}) \ \vee \ (\mathsf{type}^* = 1 \wedge x^* \in \mathcal{X})\big)\end{array}\right] \leq \mathsf{negl}(\lambda).$$

where $\mathcal{O}$ *initializes the accumulator state using* Eval*, executes* Update *to update the accumulator state* $(\mathsf{acc}_{\mathcal{X}}, \mathsf{aux})$*, and executes* $\mathsf{WitCreate}$ *to return membership and non-membership witnesses for members and non-members, respectively.*

4 Lattice-based Dynamic Universal Accumulator

Our accumulator is based on Peikert et al.'s vector commitment [21]. We use a b-ary tree ($b \geq 2$) of height $h = \log_b N$, i.e., N is also the number of leaves. The accumulator hashes the N inputs $\mathbf{x}_i$ for $i \in [0, N-1]$ to the leaf values $\mathbf{m}_i$, $i \in [0, N-1]$ and then using the Verkle tree to accumulate them into a root value $\mathbf{c}$, as shown in Fig. 2. An opening proof for the message at the index between $[0, N-1]$ consists of the sequence of opening witnesses for the nodes along the path from the root to the corresponding leaf. We now define the algorithms of the proposed DUA with Fig. 1 specifying the common parameters.

1. Prime number q	▷ Modulus
2. Positive integer d	▷ M-SIS rank
3. Positive integer $b > 2$	▷ Branching factor
4. Positive integer $h > 1$	▷ Tree height
5. $w = \lceil d \log_2 q \rceil$	▷ Gadget dimension
6. $\bar{m} = \lceil 2d \log q \rceil$	▷ Trapdoor dimension
7. $m \leftarrow \bar{m} + w$	▷ Total dimension
8. $\mathbf{G} \in \mathcal{R}_q^{d \times w}$.	▷ Gadget matrix
19. $s = O(\sqrt{m}) \cdot \omega(\sqrt{\log n})$	▷ Gaussian parameter
10. Injective map $i \in [b+1] \mapsto \mathbf{H}_i \in \mathcal{R}_q^{d \times d}$ such that $\mathbf{H}_i \neq \mathbf{H}_{i'}$ for $i \neq i'$.	▷ Tag matrices
11. $\gamma \leftarrow O\big(s\sqrt{(m+wb)wb}\big)$.	▷Norm of proof

Fig. 1. Common Parameters of the Proposed Accumulator

Members vs. Non-members. The accumulator maintains a set of member secrets $\mathcal{X} = \{\mathbf{x}_{\bar{j}}\}_{\bar{j}=0}^{N-1}$, where $N = b^h$, and each member $\mathbf{x}_{\bar{j}} \in S_{\mathsf{bin}}^w$ is a binary vector. In our construction, non-members are represented by a non-binary encoding $\mathbf{z} = \mathbf{x} + \mathbf{e}$, where $\mathbf{x} \in S_{\mathsf{bin}}^w$ and $\mathbf{e} = (e, \ldots, e, e) \in \mathcal{R}^w$ with $\|e\| > 2$. The accumulator commits to the current member set $\mathcal{X}$.

– $(\mathsf{pk}_{\mathsf{acc}}, \mathsf{sk}_{\mathsf{acc}}) \leftarrow \mathsf{Gen}(1^\lambda, N)$: The algorithm generates the keys.
  1. Sample $\bar{\mathbf{A}} \leftarrow U(\mathcal{R}_q^{d \times \bar{m}})$ and $\mathbf{B} \leftarrow U(\mathcal{R}_q^{d \times w})$.
  2. Compute $(\mathbf{A}, \mathbf{T}) \leftarrow \mathsf{TrapGen}(\bar{\mathbf{A}}, \mathbf{H}_b)$, where $\mathbf{A} \in \mathcal{R}_q^{d \times m}$
  3. For each $i \in [0, b-1]$, define $\mathbf{A}_i \leftarrow \mathbf{A} - [0 \mid \mathbf{H}_i \mathbf{G}] \in \mathcal{R}_q^{d \times m}$.
  4. Sample $\mathbf{U} = [\mathbf{U}_0 | \mathbf{U}_1 | \cdots | \mathbf{U}_{b-1}] \in \mathcal{R}_q^{d \times wb}$, where each $\mathbf{U}_i \leftarrow U(\mathcal{R}_q^{d \times w})$.
  5. For $i \in [0, b-1]$, set $\mathbf{R}_{i,i} = \mathbf{0}$; For $j \neq i$, sample $\mathbf{R}_{i,j} \leftarrow \mathsf{SamplePre}(\mathbf{T}, \mathbf{A}_i, \mathbf{U}_j, s)$ such that $\mathbf{A}_i \mathbf{R}_{i,j} = \mathbf{U}_j$.
  6. Set $\mathbf{R} := \{\mathbf{R}_{i,j}\}_{i,j \in [0,b-1]} \in (\mathcal{R}^{m \times w})^{b \times b}$.
  7. Initialize $\mathbf{U}^{(1)} := \mathbf{U} \in \mathcal{R}_q^{d \times wb}$ and $\mathbf{S}^{(1)} := \mathbf{I}_{wb} \in \mathcal{R}^{wb \times wb}$.
  8. For $k \in [2, h]$, compute $\mathbf{S}^{(k)} \leftarrow \mathbf{I}_b \otimes \mathbf{G}^{-1}(\mathbf{U}^{(k-1)}) \in \mathcal{R}^{wb \times wb^k}$ and $\mathbf{U}^{(k)} \leftarrow \mathbf{U}\mathbf{S}^{(k)} \in \mathcal{R}_q^{d \times wb^k}$.
  9. Output $\mathsf{pk}_{\mathsf{acc}} = \big(\mathbf{A}, \mathbf{B}, \big(\mathbf{U}^{(k)}\big)_{k=1}^{h}, \mathbf{e}\big)$ and $\mathsf{sk}_{\mathsf{acc}} = \mathbf{R}$.
– $(\mathsf{acc}_{\mathcal{X}}, \mathsf{aux}) \leftarrow \mathsf{Eval}(\mathsf{pk}_{\mathsf{acc}}, \mathcal{X})$: Let $\mathcal{X} = (\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_{N-1}) \in (S_{\mathsf{bin}}^w)^N$ be the set of values to be accumulated. The accumulation algorithm does:

1. For each $\bar{j} \in [0, N-1]$, compute $\mathbf{y}_{\bar{j}} \leftarrow \mathbf{B}\mathbf{x}_{\bar{j}}$ and $\mathbf{m}_{\bar{j}} \leftarrow \mathbf{G}^{-1}(\mathbf{y}_{\bar{j}})$.
2. Set $\bar{\mathbf{m}} = (\mathbf{m}_0, \mathbf{m}_1, \ldots, \mathbf{m}_{N-1}) \in (S^w_{\mathsf{bin}})^N$ and $\mathbf{c} = \mathbf{G}^{-1}(\mathbf{U}^{(h)})\bar{\mathbf{m}}$.
3. Output $\mathsf{acc}_{\mathcal{X}} := \mathbf{c}$ and $\mathsf{aux} := \mathcal{X}$.

– $(\mathsf{acc}_{\mathcal{X}'}, \mathsf{aux}') \leftarrow \mathsf{Update}(\mathsf{pk}_{\mathsf{acc}}, \mathsf{acc}_{\mathcal{X}}, \mathsf{aux}, \mathbf{x}_{\bar{j}}, \mathbf{x}'_{\bar{j}})$: The algorithm updates the leaf value at index $\bar{j}$ from $\mathbf{x}_{\bar{j}}$ to $\mathbf{x}'_{\bar{j}}$, and the root value.

1. Compute $\mathbf{y}'_{\bar{j}} \leftarrow \mathbf{B}\mathbf{x}'_{\bar{j}}$ and $\mathbf{m}'_{\bar{j}} \leftarrow \mathbf{G}^{-1}(\mathbf{y}'_{\bar{j}})$.
2. For each $k = h, \ldots, 1$, write $\bar{j} = j_{k-1}b^{k-1} + \bar{j}'$, where $j_{k-1} \in [0, b-1]$ and $\bar{j}' \in [0, b^{k-1}-1]$ for $\bar{j} \in [0, b^k - 1]$.
3. Let $\bar{j} = j_{h-1}b^{h-1} + j_{h-2}b^{h-2} + \ldots + j_1 b + j_0$; Compute

$$\mathbf{U}^{(h)}_{\bar{j}} = \mathbf{U}_{j_{h-1}}\mathbf{G}^{-1}\Big(\mathbf{U}_{j_{h-2}}\mathbf{G}^{-1}\big(\cdots \mathbf{G}^{-1}\big(\mathbf{U}_{j_1}\mathbf{G}^{-1}(\mathbf{U}_{j_0})\big)\cdots\big)\Big)$$

   where $j_{k-1} \in [0, b-1]$, $k \in [h]$, $\mathbf{U}_{j_{k-1}} \in \{\mathbf{U}_0, \ldots, \mathbf{U}_{b-1}\}$.
4. Set $\Delta\mathbf{m}_{\bar{j}} := (\mathbf{m}'_{\bar{j}} - \mathbf{m}_{\bar{j}})$, $\tilde{\mathbf{c}} \leftarrow \mathbf{G}^{-1}(\mathbf{U}^{(h)}_{\bar{j}})\,(\Delta\mathbf{m}_{\bar{j}})$, and $\mathbf{c}' \leftarrow \mathbf{c} + \tilde{\mathbf{c}}$.
5. Update the $\mathbf{x}_{\bar{j}}$'s public key as $\mathbf{y} \leftarrow \mathbf{B}(\mathbf{x}_{\bar{j}} + \mathbf{e})$.
6. Output $\mathsf{acc}_{\mathcal{X}'} := \mathbf{c}'$, and $\mathsf{aux}' := \big(\mathcal{X} \setminus \{\mathbf{x}_{\bar{j}}\}\big) \cup \{\mathbf{x}'_{\bar{j}}\}$.

– $\mathsf{wit}_x \leftarrow \mathsf{WitCreate}((\mathsf{sk}_{\mathsf{acc}}, \mathsf{pk}_{\mathsf{acc}}), \mathsf{acc}_{\mathcal{X}}, \mathsf{aux}, \mathbf{x}, \mathsf{type})$: The algorithm outputs a witness for the element $\mathbf{x}$. Let $\mathcal{X} = (\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_{N-1}) \in (S^w_{\mathsf{bin}})^N$.

If $\mathsf{type} = 0$ (membership): The algorithm does:

1. If $\mathbf{x} \notin \mathcal{X}$, return $\perp$; otherwise, find $\bar{i} \in [0, N-1]$ such that $\mathbf{x} = \mathbf{x}_{\bar{i}}$; For each $\bar{j} \in [0, N-1]$, compute $\mathbf{y}_{\bar{j}} \leftarrow \mathbf{B}\mathbf{x}_{\bar{j}}$ and $\mathbf{m}_{\bar{j}} \leftarrow \mathbf{G}^{-1}(\mathbf{y}_{\bar{j}})$.
2. Set $\bar{\mathbf{m}} = (\mathbf{m}_0, \mathbf{m}_1, \ldots, \mathbf{m}_{N-1}) \in (S^w_{\mathsf{bin}})^N$.
3. For $k \in [2, h]$, compute $\mathbf{S}^{(k)} \leftarrow \mathbf{I}_b \otimes \mathbf{G}^{-1}(\mathbf{U}^{(k-1)}) \in \mathcal{R}^{wb \times wb^k}$.
4. For $k = h$ to 2, run $\mathsf{Open}_k\big((\mathsf{sk}_{\mathsf{acc}}, \mathsf{pk}_{\mathsf{acc}}), \bar{\mathbf{m}} \in S^{wb^k}_{\mathsf{bin}}, \bar{i} \in [0, b^k - 1]\big)$ as:
   (a) Write $\bar{i} = ib^{k-1} + \bar{i}'$, where $i \in [0, b-1]$ and $\bar{i}' \in [0, b^{k-1}-1]$.
   (b) Parse $\bar{\mathbf{m}} = (\bar{\mathbf{m}}_0, \cdots, \bar{\mathbf{m}}_{b-1})$, where each $\bar{\mathbf{m}}_i \in S^{wb^{k-1}}_{\mathsf{bin}}$.
   (c) Set $\mathbf{S}^{(k)}\bar{\mathbf{m}} = (\bar{\mathbf{c}}_0, \cdots, \bar{\mathbf{c}}_{b-1}) \in S^{wb}_{\mathsf{bin}}$, and $\mathbf{p}_i = \sum_{j=0, j\neq i}^{b-1} \mathbf{R}_{i,j}\,\bar{\mathbf{c}}_j$
   (d) Set $\mathsf{Commit}_{k-1}(\mathsf{pk}_{\mathsf{acc}}, \bar{\mathbf{m}}_i)$ which does $\bar{\mathbf{c}}_i \leftarrow \mathbf{G}^{-1}(\mathbf{U}^{(k-1)})\bar{\mathbf{m}}_i$.
   (e) Set $W_{k,\bar{i}} \leftarrow (\mathbf{p}_i, \bar{\mathbf{c}}_i)$
   (f) Update $\bar{\mathbf{m}} \leftarrow \bar{\mathbf{m}}_i$, and $\bar{i} \leftarrow \bar{i}'$
5. When $k = 1$, run $\mathsf{Open}_1\big((\mathsf{sk}_{\mathsf{acc}}, \mathsf{pk}_{\mathsf{acc}}), \bar{\mathbf{m}} \in S^{wb}_{\mathsf{bin}}, \bar{i} \in [0, b-1]\big)$:
   (a) Set $i \leftarrow \bar{i}$ and parse $\bar{\mathbf{m}} = (\bar{\mathbf{m}}_0, \cdots, \bar{\mathbf{m}}_{b-1})$, where each $\bar{\mathbf{m}}_i \in S^w_{\mathsf{bin}}$.
   (b) Compute $\mathbf{p}_i = \sum_{j=0, j\neq i}^{b-1} \mathbf{R}_{i,j}\,\bar{\mathbf{m}}_j$.
   (c) Compute $\mathbf{y}_i = \mathbf{B}\mathbf{x}_i$ and $\bar{\mathbf{m}}_i \leftarrow \mathbf{G}^{-1}(\mathbf{y}_i)$.
   (d) Set $W_{1,\bar{i}} \leftarrow (\mathbf{p}_i, \bar{\mathbf{m}}_i)$
6. Output $\mathsf{wit}_x = \big(\mathbf{x}, \{W_{k,\bar{i}}\}_{k\in[1,h], \bar{i}\in[0,b^k-1]}\big)$.

If $\mathsf{type} = 1$ (non-membership), the algorithm does:

1. If $\mathbf{x} \in \mathcal{X}$, return $\perp$, else
2. Set $\mathbf{z} \leftarrow (\mathbf{x} + \mathbf{e})$, $\mathbf{y} \leftarrow \mathbf{B}\mathbf{z} \pmod q$
3. Output $\mathsf{wit}_x = (\mathbf{x}, \mathbf{y})$

– $\{\mathsf{true}, \mathsf{false}\} \leftarrow \mathsf{Verify}(\mathsf{pk}_{\mathsf{acc}}, \mathsf{acc}_{\mathcal{X}}, \mathsf{wit}_x, \mathbf{x}, \mathsf{type})$: The algorithm verifies that $\mathbf{x}$ and its witness form a valid membership (or non-membership) proof.

If $\mathsf{type} = 0$ (membership): The algorithm does:

1. Parse $\mathsf{acc}_{\mathcal{X}} = \mathbf{c}$ and $\mathsf{wit}_x = \big(\mathbf{x}, \{W_{k,\bar{i}}\}_{k\in[1,h],\bar{i}\in[0,b^k-1]}\big)$; set $\bar{\mathbf{c}} \leftarrow \mathbf{c}$.
2. For $k = h$ to 2, run $\mathsf{Verify}_k(\mathsf{pk}_{\mathsf{acc}}, \bar{\mathbf{c}}, \bar{i} \in [0, b^k - 1], \bar{\mathbf{m}}_{\bar{i}}, W_{k,\bar{i}})$:
   (a) Write $\bar{i} = ib^{k-1} + \bar{i}'$, where $i \in [0, b-1]$ and $\bar{i}' \in [0, b^{k-1} - 1]$.
   (b) Parse $(\mathbf{p}_i, \bar{\mathbf{c}}_i) \leftarrow W_{k,\bar{i}}$
   (c) Check if $\mathbf{G}\bar{\mathbf{c}} = \mathbf{A}_i\mathbf{p}_i + \mathbf{U}_i\bar{\mathbf{c}}_i$ and $\|\mathbf{p}_i\| \le \gamma$; Return false if not
   (d) Update $\bar{\mathbf{c}} \leftarrow \bar{\mathbf{c}}_i$, and $\bar{i} \leftarrow \bar{i}'$, continue,
3. When $k = 1$, run $\mathsf{Verify}_1(\mathsf{pk}_{\mathsf{acc}}, \bar{\mathbf{c}}, \bar{i}, \bar{\mathbf{m}}_{\bar{i}}, W_{1,\bar{i}})$ where $\bar{i} \in [0, b-1]$:
   (a) Set $i \leftarrow \bar{i}$, parse $(\mathbf{p}_i, \bar{\mathbf{m}}_i) \leftarrow W_{1,\bar{i}}$
   (b) Check if $\mathbf{G}\bar{\mathbf{c}} = \mathbf{A}_i\mathbf{p}_i + \mathbf{U}_i\bar{\mathbf{m}}_i$ and $\|\mathbf{p}_i\| \le \gamma$; If not return false
   (c) Check $\mathbf{G}\bar{\mathbf{m}}_i = \mathbf{y}_i$ where $\mathbf{y}_i = \mathbf{B}\mathbf{x}_i$; If not return false; Else true

If $\mathsf{type} = 1$ (non-membership): The algorithm does:

1. Parse the witness as $\mathsf{wit}_x = (\mathbf{x}, \mathbf{y})$.
2. If $\mathbf{y} - \mathbf{B}\mathbf{e} = \mathbf{B}\mathbf{x} \pmod q$ and $\mathbf{x} \in S^w_{\mathsf{bin}}$, return true; Else, return false

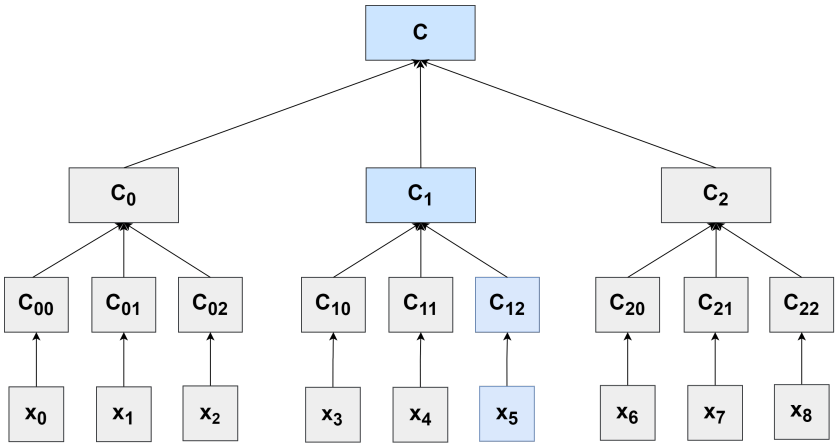


Fig. 2. A 3-ary Verkle tree with $3^2 = 9$ leaves that accumulates the set $\mathcal{X} = \{\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_8\}$ into the value $\mathbf{c}$ at the root. The blue nodes show the Verkle proof path for the element $\mathbf{x}_5$ with index string (12), which proves $\mathbf{x}_5$ is accumulated in $\mathbf{c}$.

Lemma 1 (Correctness). *The accumulator system of Section 4 is correct.*

Proof. We consider the cases for membership/non-membership witness.

Case 1*: Non-membership* ($\mathsf{type} = 0$). Let $\mathbf{x} \in \mathcal{X}$ and let wit_x be a correctly generated membership witness for $\mathbf{x}$ with respect to the accumulator value $(\mathsf{acc}_{\mathcal{X}}, \mathsf{aux})$. Let an index $\bar{i} \in [0, N-1]$ denote the position corresponding to $\mathbf{x}$.

The witness is of the form $\mathsf{wit}_x = \big(\mathbf{x}, \{W_{k,\bar{i}}\}_{k\in[1,h],\bar{i}\in[0,b^k-1]}\big)$.

The opening procedure recursively follows the path for $k = h, h-1, \ldots, 1$.

- For $k = h$, we consider Open_h. Let $\bar{i} \in [0, b^h - 1]$ be the index of $\mathbf{x}$, and write $\bar{i} = i\, b^{h-1} + \bar{i}'$, where $i \in [0, b-1]$, $\bar{i}' \in [0, b^{h-1} - 1]$.
  Since the root accumulator is computed directly from all leaves, we parse $\bar{\mathbf{m}} = (\bar{\mathbf{m}}_0, \ldots, \bar{\mathbf{m}}_{b-1})$, where each $\bar{\mathbf{m}}_i \in S_{\mathsf{bin}}^{wb^{h-1}}$ is the i-th subtree below the root. Thus Open_h outputs a proof for the child indexed by i. We then proceed to Open_{h-1} with $\bar{\mathbf{m}} \leftarrow \bar{\mathbf{m}}_i$, and $\bar{i} \leftarrow \bar{i}'$.
- For $k = h-1$, we consider Open_{h-1}. Let $\bar{i} \in [0, b^{h-1}-1]$ and write $\bar{i} = i\, b^{h-2} + \bar{i}'$, where $i \in [0, b-1]$, $\bar{i}' \in [0, b^{h-2}-1]$. We parse $\bar{\mathbf{m}} = (\bar{\mathbf{m}}_0, \ldots, \bar{\mathbf{m}}_{b-1})$, where each $\bar{\mathbf{m}}_i \in S_{\mathsf{bin}}^{wb^{h-2}}$. Thus Open_{h-1} outputs a proof for the child indexed by i. We then proceed to Open_{h-2} with $\bar{\mathbf{m}} \leftarrow \bar{\mathbf{m}}_i$, and $\bar{i} \leftarrow \bar{i}'$.
- Generally, for each $k \in [h]$, we consider Open_k. Let $\bar{i} \in [0, b^k - 1]$ and write $\bar{i} = i\, b^{k-1} + \bar{i}'$, where $i \in [0, b-1]$, $\bar{i}' \in [0, b^{k-1} - 1]$. We parse $\bar{\mathbf{m}} = (\bar{\mathbf{m}}_0, \ldots, \bar{\mathbf{m}}_{b-1})$, where each $\bar{\mathbf{m}}_i \in S_{\mathsf{bin}}^{wb^{k-1}}$.

We now consider the opening and verification procedure at stage k. Note that $\bar{i} \in [0, b^k - 1]$ denote the index of the opened leaf, and $\bar{\mathbf{m}} = (\bar{\mathbf{m}}_0, \ldots, \bar{\mathbf{m}}_{b-1})$, where each $\bar{\mathbf{m}}_i \in S_{\mathsf{bin}}^{wb^{k-1}}$ is the message of the i-th child subtree. For ith child subtree, let $\bar{\mathbf{c}}_i \leftarrow \mathsf{Commit}_{k-1}(\mathsf{pk}_{\mathsf{acc}}, \bar{\mathbf{m}}_i)$. Then the parent commitment is $\bar{\mathbf{c}}$.

The opening algorithm Open_k outputs the witness $W_{k,\bar{i}} = (\mathbf{p}_i, \bar{\mathbf{c}}_i)$ where $\mathbf{p}_i$ is the opening proof showing that $\bar{\mathbf{c}}_i$ is the correct i-th child under the parent commitment $\bar{\mathbf{c}}$. By the definition of the proof, $\mathbf{p}_i = \sum_{j=0, j\neq i}^{b-1} \mathbf{R}_{i,j}\bar{\mathbf{c}}_j$. Hence,

$$\mathbf{A}_i\mathbf{p}_i = \mathbf{A}_i \sum\nolimits_{j=0,j\neq i}^{b-1} \mathbf{R}_{i,j}\bar{\mathbf{c}}_j = \sum\nolimits_{j=0,j\neq i}^{b-1} (\mathbf{A}_i\mathbf{R}_{i,j})\bar{\mathbf{c}}_j = \sum\nolimits_{j=0,j\neq i}^{b-1} \mathbf{U}_j\bar{\mathbf{c}}_j.$$

Since $\mathbf{A}_i\mathbf{R}_{i,j} = \mathbf{U}_j$ for $j \neq i$ and $\mathbf{R}_{i,i} = 0$, we have

$$\mathbf{A}_i\mathbf{p}_i + \mathbf{U}_i\bar{\mathbf{c}}_i = \sum\nolimits_{j=0}^{b-1} \mathbf{U}_j\bar{\mathbf{c}}_j = \mathbf{G}\bar{\mathbf{c}}$$

as required. Furthermore, let $\bar{\mathbf{u}}^\top := \big[\bar{\mathbf{c}}_0^\top, \cdots, \bar{\mathbf{c}}_{b-1}^\top\big] \in \mathcal{R}^{1\times wb}$, and $\mathbf{R}_{i,*} := [\mathbf{R}_{i,0}, \cdots, \mathbf{R}_{i,b-1}]$. Then $\mathbf{p}_i = \sum_{j=0,j\neq i}^{b-1} \mathbf{R}_{i,j}\,\bar{\mathbf{c}}_j$. Hence, $\|\mathbf{p}_i\| \le s_1(\mathbf{R}_{i,*})\,\|\bar{\mathbf{u}}\|$. Since $\bar{\mathbf{u}} \in S_{\mathsf{bin}}^{wb}$, we have $\|\bar{\mathbf{u}}\| \le \sqrt{wb}$. Moreover, by Item 3b of Theorem 1, $s_1(\mathbf{R}_{i,*}) = O\left(s\sqrt{m+wb}\right)$. Therefore, $\|\mathbf{p}_i\| \le O\big(s\sqrt{m+wb}\big)\sqrt{wb} = O\Big(s\sqrt{(m+wb)\,wb}\Big)$. Let $\gamma \leftarrow O\Big(s\sqrt{(m+wb)\,wb}\Big)$. Thus, we have $\|\mathbf{p}_i\| \le \gamma$. Hence Verify_k accepts: $\mathbf{G}\bar{\mathbf{c}} = \mathbf{A}_i\mathbf{p}_i + \mathbf{U}_i\bar{\mathbf{c}}_i$ and $\|\mathbf{p}_i\| \le \gamma$. Similarly, every valid proof produced by Open_k is accepted by Verify_k for each $k = h, \cdots, 2$.

We now consider the opening and verification procedure at $k = 1$. Here we consider Open_1. Let $\bar{i} \in [0, b-1]$ and parse $\bar{\mathbf{m}} = (\bar{\mathbf{m}}_0, \ldots, \bar{\mathbf{m}}_{b-1})$, where each $\bar{\mathbf{m}}_i \in S_{\mathsf{bin}}^{w}$ is a leaf value. The opening algorithm Open_1 outputs the witness

$W_{1,\bar{i}} = (\mathbf{p}_i, \bar{\mathbf{m}}_i)$, for the position $\bar{i} \in [0, b-1]$ (Note that when $k = 1$, $\bar{i} = i$). Since the base accumulator verification checks that $\mathbf{G}\bar{\mathbf{c}} = \mathbf{A}_i\mathbf{p}_i + \mathbf{U}_i\bar{\mathbf{m}}_i$ and $\|\mathbf{p}_i\| \leq \gamma$, the claim follows exactly as in the above proof for the case $k = 1$. Furthermore, Verify_1 proves the leaf indexed by $\bar{i}$, which corresponds to $\mathbf{x}$. Note that in the bottom layer we have $\bar{\mathbf{m}}_i$ where $\mathbf{m}_i = \mathbf{G}^{-1}(\mathbf{y}_i)$ for $\mathbf{y}_i = \mathbf{B}\mathbf{x}_i$. Hence, every valid proof produced by Open_1 is accepted by Verify_1 at $k = 1$.

Since all verification equations hold for $k = h, \ldots, 1$, the verifier accepts every honestly generated membership witness.

Case 2*: Non-membership* ($\mathsf{type} = 1$). Let the witness $\mathsf{wit}_x = (\mathbf{x}, \mathbf{y})$ for $\mathbf{x} \notin \mathcal{X}$. Assume the witness is honestly generated from a binary vector $\mathbf{x} \in S^w_{\mathsf{bin}}$ and a public vector $\mathbf{e} = (e, \ldots, e, e) \in \mathcal{R}^w$ where $\|e\| > 2$. Let $\mathbf{z} := \mathbf{x} + \mathbf{e}$ and $\mathbf{y} := \mathbf{B}\mathbf{z} = \mathbf{B}(\mathbf{x} + \mathbf{e}) \pmod q$. The verifier accepts iff $\mathbf{y} - \mathbf{B}\mathbf{e} = \mathbf{B}\mathbf{x} \pmod q$ where $\mathbf{x} \in S^w_{\mathsf{bin}}$. We show that these conditions hold for an honestly generated witness. Using the definition of $\mathbf{y}$,

$$\mathbf{y} - \mathbf{B}\mathbf{e} = \mathbf{B}(\mathbf{x} + \mathbf{e}) - \mathbf{B}\mathbf{e} = \mathbf{B}\mathbf{x} \pmod q.$$

Therefore, every honestly generated non-membership witness is accepted by the verifier. Thus, in both cases, membership and non-membership, any correctly generated, up-to-date witness is accepted by the verification algorithm. Therefore, our accumulator construction is correct.

4.1 Zero-Knowledge Argument of Knowledge for Accumulator

In this section, we give a ZKAoK (Definition 5) to prove the possession of the membership/non-membership witness. We show how to convert the membership/non-membership verification relations to *quadratic relations* that can be efficiently proved by the ZKAoK system due to Lyubashevsky et al.'[18, Sec. 4], demonstrating efficient zero-knowledge membership/non-membership proofs. Such a proof can be made non-interactive using the Fiat-Shamir heuristic [18].

Let $q_1 < q$ be a prime such that $q_1 \equiv 2k+1 \pmod{4k}$, and let $q_\pi = q_1 q$ denote the proof-system modulus, which is distinct from the accumulator modulus. We choose q_1 so that it has the same splitting as q in $\mathcal{R}$, ensuring that challenge differences are invertible in $\mathcal{R}_{q_\pi}$ [18, Sec. 2.3].

Case 1*: Membership* ($\mathsf{type} = 0$). The prover $\mathcal{P}$ convinces the verifier $\mathcal{V}$ that it knows $(\mathbf{x}, \mathsf{wit}_x)$ satisfying the following NP-relation:

$$\mathcal{R}_{\mathsf{mem}} = \left\{ \begin{array}{l|l} \big((\mathsf{pk}_{\mathsf{acc}}, \mathsf{acc}_{\mathcal{X}}, \mathbf{x}), & \mathsf{wit}_x = \big(\mathbf{x}, \{W_{k,\bar{i}}\}_{k\in[1,h],\bar{i}\in[0,b^k-1]}\big), \\ (\mathbf{x}, \mathsf{wit}_x)\big) & k = h, \cdots, 2: \quad W_{k,\bar{i}} \leftarrow (\mathbf{p}_i, \bar{\mathbf{c}}_i), \\ & \quad \mathbf{G}\bar{\mathbf{c}} = \mathbf{A}_i\mathbf{p}_i + \mathbf{U}_i\bar{\mathbf{c}}_i, \quad \|\mathbf{p}_i\| \leq \gamma \\ & k = 1: \quad W_{1,\bar{i}} \leftarrow (\mathbf{p}_i, \bar{\mathbf{m}}_i), \\ & \quad \mathbf{G}\bar{\mathbf{c}} = \mathbf{A}_i\mathbf{p}_i + \mathbf{U}_i\bar{\mathbf{m}}_i, \quad \|\mathbf{p}_i\| \leq \gamma \\ & \quad \mathbf{x}_i \in S^w_{\mathsf{bin}}, \quad \mathbf{y}_i = \mathbf{B}\mathbf{x}_i, \quad \mathbf{G}\bar{\mathbf{m}}_i = \mathbf{y}_i \end{array} \right\}.$$

We now show how to generate a zero-knowledge proof for the verification relation. Note that, $W_{k,\bar{i}} \leftarrow (\mathbf{p}_i, \bar{\mathbf{c}}_i)$, then the prover must prove knowledge of

$(\mathbf{p}_i, \bar{\mathbf{c}}_i)$ such that $\mathbf{A}_i\mathbf{p}_i + \mathbf{U}_i\bar{\mathbf{c}}_i = \mathbf{G}\bar{\mathbf{c}}$, $\|\mathbf{p}_i\| \leq \gamma$ for $i \in [0, b-1]$. Equivalently, the prover proves the relation

$$q_1\big(\mathbf{A}_i\mathbf{p}_i + \mathbf{U}_i\bar{\mathbf{c}}_i\big) \equiv q_1\mathbf{G}\bar{\mathbf{c}} \pmod{q_\pi\mathcal{R}} \ \wedge \ \|\mathbf{p}_i\| \leq \gamma.$$

where the private inputs are $(\mathbf{p}_i, \bar{\mathbf{c}}_i)$ and the public inputs are $(\mathbf{A}_i, \mathbf{U}_i, \bar{\mathbf{c}})$. We define $\mathbf{s}_1 = [\,\mathbf{p}_i \mid \bar{\mathbf{c}}_i\,] \in \mathcal{R}^{m+w}$ and $\mathbf{s} = [\,\mathbf{s}_1 \mid \sigma_{-1}(\mathbf{s}_1)\,] \in \mathcal{R}^{2(m+w)}$.

1. Quadratic Relation: Define $f_r(\mathbf{s}) = \mathbf{e}_r^\intercal[q_1\mathbf{A}_i|q_1\mathbf{U}_i|0_{d\times(m+w)}]\mathbf{s} + (-\mathbf{e}_r^T q_1\mathbf{G}\bar{\mathbf{c}})$ for all $r \in [d]$ where $\mathbf{e}_r$ be the zero vector with 1 at the rth position. Then proving $f_r(\mathbf{s}) = 0 \mod q_\pi\mathcal{R}$ for all $r \in [d]$ yields $q_1\big(\mathbf{A}_i\mathbf{p}_i + \mathbf{U}_i\bar{\mathbf{c}}_i\big) = q_1\mathbf{G}\bar{\mathbf{c}} \pmod{q_\pi\mathcal{R}}$.
2. Norm Conditions: Define $\mathbf{E}^e = [\mathbf{I}_m|0_{m\times(m+2w)}], \mathbf{u}^e = 0_m$, and $\beta^e = \gamma$. So, $\|\mathbf{E}^e\mathbf{s} - \mathbf{u}^e\|_2 \leq \beta^e$ is equivalent to $\|\mathbf{p}_i\| \leq \gamma$.

Note that the ZKAoK can be applied independently for each $k \in [h]$, thereby certifying the correctness of the authentication path from the root to the leaf.

At the bottom layer, the prover must additionally prove the relation $\mathbf{y}_i = \mathbf{B}\mathbf{x}_i$ where $\mathbf{x}_i \in S^w_{\mathsf{bin}}$. Equivalently, the prover proves the relation

$$q_1\mathbf{y}_i \equiv q_1\mathbf{B}\mathbf{x}_i \pmod{q_\pi\mathcal{R}} \ \wedge \ \mathbf{x}_i \in S^w_{\mathsf{bin}}$$

where the private input is $\mathbf{x}_i$ and the public inputs are $(\mathbf{B}, \mathbf{y}_i)$. We define $\mathbf{s}_1 = [\,\mathbf{x}_i\,] \in \mathcal{R}^w$ and $\mathbf{s} = [\,\mathbf{s}_1 \mid \sigma_{-1}(\mathbf{s}_1)\,] \in \mathcal{R}^{2w}$.

1. Quadratic Relation: Define $f_r(\mathbf{s}) = \mathbf{e}_r^\intercal[q_1\mathbf{B} \mid 0_{d\times w}]\mathbf{s} + (-\mathbf{e}_r^T q_1\mathbf{y}_i)$ for all $r \in [d]$ where $\mathbf{e}_r$ be the zero vector with 1 at the rth position. Then proving $f_r(\mathbf{s}) = 0 \mod q_\pi\mathcal{R}$ for all $r \in [d]$ yields $q_1\mathbf{B}\mathbf{x}_i = q_1\mathbf{y}_i \pmod{q_\pi\mathcal{R}}$.
2. $\mathbf{x}$ is binary: Define $\mathbf{r} = \sum_{j\in[0,n-1]}\mathbf{X}^j$ and $F_r(\mathbf{s}) = \mathbf{s}^\intercal\mathbf{E}_{r,r}\mathbf{s} + (-\mathbf{r}\mathbf{e}_r)^\intercal\mathbf{s} = \sigma_{-1}(\mathbf{x}_i[r])(\mathbf{x}_i[r] - \mathbf{r}) \ \ \forall r \in [w]$ where $\mathbf{E}_{k,\ell}$ is the zero matrix with 1 at position (k, ℓ). Here $\tilde{F}_r(\mathbf{s}) = 0$ for all $r \in [w]$ yields $\mathbf{x}_i \in S^w_{\mathsf{bin}}$.

Case 2: Non-membership ($\mathsf{type} = 1$). The prover $\mathcal{P}$ convinces the verifier $\mathcal{V}$ that it knows $(\mathbf{x}, \mathsf{wit}_x)$ satisfying the following NP-relation:

$$\mathcal{R}_{\mathsf{non\text{-}mem}} = \left\{ \big((\mathsf{pk}_{\mathsf{acc}}, \mathsf{acc}_{\mathcal{X}}, \mathbf{x}), (\mathbf{x}, \mathsf{wit}_x)\big) \;\middle|\; \begin{array}{l} \mathsf{wit}_x = (\mathbf{x}, \mathbf{y}), \mathbf{x} \in S^w_{\mathsf{bin}}, \\ \mathbf{y} - \mathbf{B}\mathbf{e} = \mathbf{B}\mathbf{x} \pmod{q} \end{array} \right\}.$$

We now show how to generate a zero-knowledge proof for the verification relation. Equivalently, the prover must prove the relation

$$q_1(\mathbf{y} - \mathbf{B}\mathbf{e}) \equiv q_1\mathbf{B}\mathbf{x} \pmod{q_\pi\mathcal{R}} \ \wedge \ \mathbf{x} \in S^w_{\mathsf{bin}}$$

where the private input is $(\mathbf{x})$ and the public inputs are $(\mathbf{B}, \mathbf{e}, \mathbf{y})$. We define $\mathbf{s}_1 = [\,\mathbf{x}\,] \in \mathcal{R}^w$ and $\mathbf{s} = [\,\mathbf{s}_1 \mid \sigma_{-1}(\mathbf{s}_1)\,] \in \mathcal{R}^{2w}$.

1. Quadratic Relation: Define $f_r(\mathbf{s}) = \mathbf{e}_r^\intercal[q_1\mathbf{B} \mid 0_{d\times w}]\mathbf{s} + (-\mathbf{e}_r^T q_1\mathbf{y} - \mathbf{e}_r^T q_1\mathbf{B}\mathbf{e})$ for all $r \in [d]$ where $\mathbf{e}_r$ be the zero vector with 1 at the rth position. Then proving $f_r(\mathbf{s}) = 0 \mod q_\pi\mathcal{R}$ for all $r \in [d]$ yields $q_1\mathbf{B}\mathbf{x} = q_1(\mathbf{y} - \mathbf{B}\mathbf{e}) \pmod{q_\pi\mathcal{R}}$.
2. $\mathbf{x}$ is binary: Define $\mathbf{r} = \sum_{j\in[0,n-1]}\mathbf{X}^j$ and $F_r(\mathbf{s}) = \mathbf{s}^\intercal\mathbf{E}_{r,r}\mathbf{s} + (-\mathbf{r}\mathbf{e}_r)^\intercal\mathbf{s} = \sigma_{-1}(\mathbf{x}[r])(\mathbf{x}[r] - \mathbf{r}) \ \ \forall r \in [w]$ where $\mathbf{E}_{k,\ell}$ is the zero matrix with 1 at position (k, ℓ). Here $\tilde{F}_r(\mathbf{s}) = 0$ for all $r \in [w]$ yields $\mathbf{x} \in S^w_{\mathsf{bin}}$.

4.2 Security Analysis

Theorem 2. *The accumulator system of Section 4 is collision resistant.*

Proof. Assume, for contradiction, that there exists a p.p.t. adversary $\mathcal{A}$ that breaks the collision resistance of DUA with non-negligible advantage $\epsilon(\lambda)$, where $\epsilon(\lambda)$ denotes the success probability of $\mathcal{A}$ in Definition 3. Any successful forgery by $\mathcal{A}$ falls into one of two cases: the membership or non-membership. Let $\epsilon_0(\lambda)$ and $\epsilon_1(\lambda)$ denote the corresponding success probabilities, so $\epsilon(\lambda) = \epsilon_0(\lambda) + \epsilon_1(\lambda)$.

Case 1: Membership ($\mathsf{type} = 0$). Assume that $\epsilon_0(\lambda)$ is non-negligible. We show how to construct an algorithm $\mathcal{B}_0$ that breaks the position-binding property of the underlying vector commitment scheme VC. First, $\mathcal{B}_0$ runs $(cp, vp) \leftarrow \mathsf{Setup}(\lambda, N)$ of Peikert et. al [21], where $cp = (\mathbf{U}, \mathbf{R})$ and $vp = (\mathbf{A}, \mathbf{U})$. It then samples $\mathbf{B} \leftarrow U(\mathcal{R}_q^{d \times w})$ and sets $\mathbf{e} = (e, \ldots, e, e) \in \mathcal{R}^w$ where $\|e\| > 2$. Next, $\mathcal{B}_0$ defines $\mathsf{pk}_{\mathsf{acc}} = \big(\mathbf{A}, \mathbf{B}, \mathbf{U}, \mathbf{e}\big)$ and $\mathsf{sk}_{\mathsf{acc}} = \mathbf{R}$. Finally, $\mathcal{B}_0$ sends $\mathsf{pk}_{\mathsf{acc}}$ to $\mathcal{A}$.

$\mathcal{B}_0$ answers to oracle queries as follows:

- For $\mathcal{O}_{\mathsf{Eval}}()$: On input $\mathcal{X} = \{\mathbf{x}_{\bar{j}}\}_{\bar{j} \in [0, N-1]}$, compute $\mathbf{m}_{\bar{j}} \leftarrow \mathbf{G}^{-1}(\mathbf{B}\mathbf{x}_{\bar{j}})$ for all $\bar{j} \in [0, N-1]$, $\bar{\mathbf{m}} \leftarrow (\mathbf{m}_0, \ldots, \mathbf{m}_{N-1})$, $\mathbf{c} \leftarrow \mathsf{Commit}_h(cp, \bar{\mathbf{m}})$ of VC, then set $\mathsf{acc}_{\mathcal{X}} \leftarrow \mathbf{c}$, $\mathsf{aux} \leftarrow \mathcal{X}$, and return $(\mathsf{acc}_{\mathcal{X}}, \mathsf{aux})$ to $\mathcal{A}$.
- For $\mathcal{O}_{\mathsf{Update}}()$: On input $\mathbf{x}_{\bar{j}} \mapsto \mathbf{x}'_{\bar{j}}$, compute $\mathbf{y}'_{\bar{j}} \leftarrow \mathbf{B}\mathbf{x}'_{\bar{j}}$, $\mathbf{m}'_{\bar{j}} \leftarrow \mathbf{G}^{-1}(\mathbf{y}'_{\bar{j}})$, $\Delta\mathbf{m}_{\bar{j}} \leftarrow (\mathbf{m}'_{\bar{j}} - \mathbf{m}_{\bar{j}})$, $\tilde{\mathbf{c}} \leftarrow \mathbf{G}^{-1}(\mathbf{U}_{\bar{j}}^{(h)})\, \Delta\mathbf{m}_{\bar{j}}$, $\mathbf{c}' \leftarrow \mathsf{UpdateC}(vp, \mathbf{c}, \tilde{\mathbf{c}})$ of VC, set $\mathsf{acc}'_{\mathcal{X}} \leftarrow \mathbf{c}'$, $\mathsf{aux}' \leftarrow \big(\mathcal{X} \setminus \{\mathbf{x}_{\bar{j}}\}\big) \cup \{\mathbf{x}'_{\bar{j}}\}$, and return $(\mathsf{acc}'_{\mathcal{X}}, \mathsf{aux}')$ to $\mathcal{A}$.
- For $\mathcal{O}_{\mathsf{WitCreate}}()$: On input $\mathbf{x}$, if $\mathbf{x} \notin \mathcal{X}$, return $\perp$; else, compute $\mathbf{y} \leftarrow \mathbf{B}\mathbf{x}$ and $\mathbf{m} \leftarrow \mathbf{G}^{-1}(\mathbf{y})$. If $\mathbf{m} \notin \bar{\mathbf{m}}$, return $\perp$. Otherwise, identify $\bar{i} \in [0, N-1]$ such that $\mathbf{m} = \bar{\mathbf{m}}_{\bar{i}}$, compute $\bar{p}_{\bar{i}} \leftarrow \mathsf{Open}_k(cp, \bar{i} \in [0, b^k - 1], \bar{\mathbf{m}})$ of VC for each $k \in [h]$. Set $W_{k,\bar{i}} \leftarrow \bar{p}_{\bar{i}}$ return $\mathsf{wit}_x = \big(\mathbf{x}, \{W_{k,\bar{i}}\}_{k \in [1,h], \bar{i} \in [0, b^k - 1]}\big)$.

Eventually, $\mathcal{A}$ outputs a forgery $(\mathbf{x}', \mathsf{wit}'_x)$ such that $\mathbf{x}' \notin \mathcal{X}$ and $\mathsf{Verify}(\mathsf{pk}_{\mathsf{acc}}, \mathsf{acc}_{\mathcal{X}}, \mathsf{wit}'_x, \mathbf{x}', \mathsf{type} = 0) = \mathsf{true}$. Since the verification algorithm accepts, the forged witness defines a valid authentication path from the leaf indexed by $\bar{i} \in [0, N-1]$ to the root $\mathsf{acc}_{\mathcal{X}}$. Let $k \in [h]$ be the first level at which the forged path differs from the honest path for the same index, and let $\bar{\mathbf{c}}$ be the corresponding parent commitment. Parse $W'_{k,\bar{i}} = (\mathbf{p}'_i, \bar{\mathbf{c}}'_i)$ and $\bar{p}_{\bar{i}} = (\mathbf{p}_i, \bar{\mathbf{c}}_i, \cdot)$, where $\bar{p}_{\bar{i}}$ is the honest opening at that level. Then $\mathcal{B}_0$ outputs $\Big(\bar{\mathbf{c}},\ i,\ (\bar{\mathbf{c}}_i,\ \mathbf{p}_i),\ (\bar{\mathbf{c}}'_i,\ \mathbf{p}'_i)\Big)$ as its own solution to position-binding challenge of VC. As $\mathsf{Verify}(\mathsf{pk}_{\mathsf{acc}}, \mathsf{acc}_{\mathcal{X}}, \mathsf{wit}'_x, \mathbf{x}', \mathsf{type} = 0) = \mathsf{true}$ that implies $\mathbf{G}\bar{\mathbf{c}} = \mathbf{A}_i\mathbf{p}'_i + \mathbf{U}_i\bar{\mathbf{c}}'_i$ and $\|\mathbf{p}'_i\| \leq \gamma$ in VC and the pair $(\bar{\mathbf{c}}', \mathbf{p}'_i)$ is a valid opening for commitment $\bar{\mathbf{c}}$. Hence we conclude $\mathcal{B}_0$'s output gives a valid forgery to VC, contradicting to the position-binding of VC. Furthermore, if $\mathcal{A}$ succeeds with non-negligible probability $\epsilon_0(\lambda)$, then $\mathcal{B}_0$ breaks the collision-resistance security of DUA with the same probability. Since VC is position binding, it follows that $\epsilon_0(\lambda) \in \mathsf{negl}(\lambda)$.

Case 2: Non-membership ($\mathsf{type} = 1$). Assume that $\epsilon_1(\lambda)$ is non-negligible. We show how to construct an algorithm $\mathcal{B}_1$ that solves the $\mathsf{SIS}_{q,d,w,\beta}$ problem. On input $\mathbf{B} \in \mathcal{R}_q^{d \times w}$, algorithm $\mathcal{B}_1$ outputs a nonzero short vector $\mathbf{s} \in \mathcal{R}^w$ such

that $\mathbf{Bs} = 0 \pmod q$ and $\|\mathbf{s}\| \leq \beta$. First, $\mathcal{B}_1$ runs $(\mathsf{pk}_{\mathsf{acc}}, \mathsf{sk}_{\mathsf{acc}}) \leftarrow \mathsf{Gen}(1^\lambda, N)$, and then sends $\mathsf{pk}_{\mathsf{acc}}$ to $\mathcal{A}$.

$\mathcal{B}_1$ answers to oracle queries as follows:

- For $\mathcal{O}_{\mathsf{Eval}}()$: On input $\mathcal{X} = \{\mathbf{x}_{\bar{j}}\}_{\bar{j}\in[0,N-1]}$, compute $\mathbf{m}_{\bar{j}} \leftarrow \mathbf{G}^{-1}(\mathbf{B}\mathbf{x}_{\bar{j}})$ for all $\bar{j} \in [0, N-1]$, $\bar{\mathbf{m}} \leftarrow (\mathbf{m}_0, \ldots, \mathbf{m}_{N-1})$, $\bar{\mathbf{c}} = \mathbf{G}^{-1}(\mathbf{U}^{(h)})\bar{\mathbf{m}}$, $\mathbf{c} \leftarrow \tilde{\mathbf{c}}$, then set $\mathsf{acc}_{\mathcal{X}} \leftarrow \mathbf{c}$, $\mathsf{aux} \leftarrow \mathcal{X}$, and return $(\mathsf{acc}_{\mathcal{X}}, \mathsf{aux})$ to $\mathcal{A}$.
- For $\mathcal{O}_{\mathsf{Update}}()$: On input $\mathbf{x}_{\bar{j}} \mapsto \mathbf{x}'_{\bar{j}}$, compute $\mathbf{y}'_{\bar{j}} \leftarrow \mathbf{B}\mathbf{x}'_{\bar{j}}$, $\mathbf{m}'_{\bar{j}} \leftarrow \mathbf{G}^{-1}(\mathbf{y}'_{\bar{j}})$, $\Delta\mathbf{m}_{\bar{j}} \leftarrow (\mathbf{m}'_{\bar{j}} - \mathbf{m}_{\bar{j}})$, $\tilde{\mathbf{c}} \leftarrow \mathbf{G}^{-1}(\mathbf{U}^{(h)}_{\bar{j}})\,\Delta\mathbf{m}_{\bar{j}}$, $\mathbf{c}' \leftarrow \mathbf{c} + \tilde{\mathbf{c}}$, $\mathsf{acc}'_{\mathcal{X}} \leftarrow \mathbf{c}'$, $\mathsf{aux}' \leftarrow \big(\mathcal{X} \setminus \{\mathbf{x}_{\bar{j}}\}\big) \cup \{\mathbf{x}'_{\bar{j}}\}$, and return $(\mathsf{acc}'_{\mathcal{X}}, \mathsf{aux}')$ to $\mathcal{A}$.
- For $\mathcal{O}_{\mathsf{WitCreate}}()$: On input $\mathbf{x}$, if $\mathbf{x} \in \mathcal{X}$, return $\perp$; else, compute $\mathbf{z} \leftarrow (\mathbf{x} + \mathbf{e})$, $\mathbf{y} \leftarrow \mathbf{B}\mathbf{z}$, return $\mathsf{wit}_x = (\mathbf{x}, \mathbf{y})$.

Eventually, $\mathcal{A}$ outputs a forgery $(\mathbf{x}', \mathsf{wit}'_x)$ such that $\mathbf{x}' \in \mathcal{X}$ and $\mathsf{Verify}(\mathsf{pk}_{\mathsf{acc}}, \mathsf{acc}_{\mathcal{X}}, \mathsf{wit}'_x, \mathbf{x}', \mathsf{type} = 1) = \mathsf{true}$. Since the forged non-membership witness is accepted, it defines a valid relation $\mathbf{y}' = \mathbf{B}\mathbf{z}' \pmod q$, where $\mathbf{z}' = \mathbf{x}' + \mathbf{e}$. On the other hand, since $\mathbf{x}' \in \mathcal{X}$, it also has a valid membership witness in the accumulator. Therefore, for the same value $\mathbf{y}'$, we have $\mathbf{y}' = \mathbf{B}\mathbf{x}' \pmod q$. Subtracting the two equations yields $\mathbf{B}(\mathbf{z}' - \mathbf{x}') = \mathbf{B}\mathbf{e} = 0 \pmod q$. Now let $\mathbf{s} := \mathbf{z}' - \mathbf{x}' = \mathbf{e}$. Since $\mathbf{e} \neq 0$ and $\|\mathbf{e}\| \leq \beta$, it follows that $\mathbf{s}$ is a nonzero short vector satisfying $\mathbf{Bs} = 0 \pmod q$. Therefore, $\mathcal{B}_1$ outputs a valid solution to the $\mathsf{SIS}_{q,d,w,\beta}$ instance, contradicting the hardness of $\mathsf{SIS}_{q,d,w,\beta}$. Thus, if $\mathcal{A}$ succeeds with non-negligible probability $\epsilon_1(\lambda)$, then $\mathcal{B}_1$ solves $\mathsf{SIS}_{q,d,w,\beta}$ with the same probability. Since $\mathsf{SIS}_{q,d,w,\beta}$ is hard, it follows that $\epsilon_1(\lambda) \in \mathsf{negl}(\lambda)$.

Therefore, the success probability in each case is negligible, i.e., $\epsilon_0(\lambda), \epsilon_1(\lambda) \in \mathsf{negl}(\lambda)$. Since $\epsilon(\lambda) = \epsilon_0(\lambda) + \epsilon_1(\lambda)$, we conclude that $\epsilon(\lambda) \in \mathsf{negl}(\lambda)$. This contradicts the assumption that $\mathcal{A}$ succeeds with non-negligible probability. Hence, DUA is collision resistant. □

5 Conclusion

In this paper, we presented a lattice-based dynamic universal accumulator based on a b-ary Verkle tree, where b denotes the branching factor. By using vector commitments instead of hash-based authentication, together with a higher branching factor, the proposed construction achieves a more scalable tree structure and more efficient membership verification than existing lattice-based Merkle tree accumulators. In particular, membership witnesses contain only the values along the *root-to-leaf* path and exclude sibling information. This *sibling-free* property makes the construction especially suitable for applications such as membership revocation in anonymous credential systems, where limiting the disclosure of auxiliary path information is important. Furthermore, the scheme supports *stateless updates*. In addition, the linearity of the vector commitment allows updates to be applied directly to the root, unlike prior lattice-based tree accumulators, which require logarithmic-time update procedures. Overall, our results show that the Verkle-tree-based design offers a practical and efficient post-quantum alternative for dynamic membership management.

References

1. Au, M.H., Tsang, P.P., Susilo, W., Mu, Y.: Dynamic universal accumulators for ddh groups and their application to attribute-based anonymous credential systems. In: CT-RSA. pp. 295–308.
2. Barthoulot, A., Blazy, O., Canard, S.: Cryptographic accumulators: new definitions, enhanced security, and delegatable proofs. In: Africacrypt 2024. pp. 94–119.
3. Benaloh, J., De Mare, M.: One-way accumulators: A decentralized alternative to digital signatures. In: EuroCrypt 1993. pp. 274–285.
4. Camenisch, J., Kohlweiss, M., Soriente, C.: An accumulator based on bilinear maps and efficient revocation for anonymous credentials. In: PKC 2009. pp. 481–500. Springer (2009)
5. Camenisch, J., Lysyanskaya, A.: Dynamic accumulators and application to efficient revocation of anonymous credentials. In: CRYPTO 2002. pp. 61–76.
6. Derler, D., Hanser, C., Slamanig, D.: Revisiting cryptographic accumulators, additional properties and relations to other primitives. In: CT-RSA 2015. pp. 127–144.
7. Gentry, C., Peikert, C., Vaikuntanathan, V.: Trapdoors for hard lattices and new cryptographic constructions. In: STOC 2008. pp. 197–206
8. Jeudy, C., Roux-Langlois, A., Sanders, O.: Lattice signature with efficient protocols, application to anonymous credentials. In: Crypto 2023. pp. 351–383.
9. Jhanwar, M.P., Safavi-Naini, R.: Compact accumulator using lattices. In: International Conference on Security, Privacy, and Applied Cryptography Engineering. pp. 347–358. Springer (2015)
10. Kemmoe, V.Y., Lysyanskaya, A., Nguyen, N.K.: Lattice-based accumulator and application to anonymous credential revocation. Cryptology ePrint Archive (2025)
11. Langlois, A., Stehlé, D.: Worst-case to average-case reductions for module lattices. Designs, Codes and Cryptography **75**(3), 565–599 (2015)
12. Li, J., Li, N., Xue, R.: Universal accumulators with efficient non-membership proofs. In: ACNS 2007. pp. 253–269.
13. Libert, B., Ling, S., Nguyen, K., Wang, H.: Zero-knowledge arguments for lattice-based accumulators: logarithmic-size ring signatures and group signatures without trapdoors. In: EUROCRYPT 2016, Part II 35. pp. 1–31.
14. Libert, B., Ling, S., Nguyen, K., Wang, H.: Zero-knowledge arguments for lattice-based PRFs and applications to E-cash. In: AsiaCrypt 2017. pp. 304–335.
15. Libert, B., Ling, S., Nguyen, K., Wang, H.: Zero-knowledge arguments for lattice-based accumulators: Logarithmic-size ring signatures and group signatures without trapdoors. Journal of Cryptology **36**(3), 23 (2023)
16. Ling, S., Nguyen, K., Wang, H., Xu, Y.: Lattice-based group signatures: achieving full dynamicity with ease. In: ACNS 2017, Proceedings 15. pp. 293–312.
17. Lyubashevsky, V., Nguyen, N.K., Plançon, M.: Lattice-based zero-knowledge proofs and applications: shorter, simpler, and more general. In: Crypto 2022. pp. 71–101.
18. Lyubashevsky, V., Nguyen, N.K., Seiler, G.: Shorter lattice-based zero-knowledge proofs via one-time commitments. In: PKC 2021. pp. 215–241.
19. Micciancio, D., Peikert, C.: Trapdoors for lattices: Simpler, tighter, faster, smaller. In:EuroCrypt 2012. pp. 700–718.
20. Miers, I., Garman, C., Green, M., Rubin, A.D.: Zerocoin: Anonymous distributed e-cash from bitcoin. In: S&P 2013. pp. 397–411. IEEE (2013)
21. Peikert, C., Pepin, Z., Sharp, C.: Vector and functional commitments from lattices. In: TCC 2021, Part III 19. pp. 480–511.

22. Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. SIAM review **41**(2), 303–332 (1999)
23. Stern, J.: A new identification scheme based on syndrome decoding. In: Crypto 1993. pp. 13–21. Springer
24. Vitto, G., Biryukov, A.: Dynamic universal accumulator with batch update over bilinear groups. In: CT-RSA 2022. pp. 395–426. Springer
25. Yang, R., Au, M.H., Zhang, Z., Xu, Q., Yu, Z., Whyte, W.: Efficient lattice-based zero-knowledge arguments with standard soundness: construction and applications. In: Crypto 2019. pp. 147–175.
26. Yu, Z., Au, M.H., Yang, R., Lai, J., Xu, Q.: Lattice-based universal accumulator with non-membership arguments. In: ACISP 2018, pp. 502–519.
27. Zhao, Y., Yang, S., Huang, X.: Lattice-based dynamic universal accumulator: Design and application. Computer Standards & Interfaces **89**, 103807 (2024)

A Interactive Zero-Knowledge Proofs

Definition 4 (Interactive Proof System). *Let* $\mathsf{R} \subseteq \mathcal{X} \times \mathcal{W}$ *be a binary relation. An interactive proof system for* R *consists of a tuple of probabilistic polynomial-time algorithms* $(\mathsf{Setup}, \mathsf{Prove}, \mathsf{Verify})$ *defined as follows:*

1. $\sigma \leftarrow \mathsf{Setup}(1^n, 1^\lambda)$*: On input the statement length* n *and the security parameter* λ*, the algorithm outputs a string* σ*.*
2. $\pi \leftarrow \mathsf{Prove}(\sigma, x, w)$*: On input a string* σ *and a statement–witness pair* $(x, w) \in \mathsf{R}$*, the algorithm outputs a proof* π*.*
3. $b \leftarrow \mathsf{Verify}(\sigma, x, \pi)$*: On input a string* σ*, a statement* x*, and a proof* π*, the algorithm outputs a bit* $b \in \{0, 1\}$*.*

Definition 5 (Zero-Knowledge Arguments of Knowledge (ZKAoK [25])). *Let* $\mathsf{R} = \{(x, w)\} \in \{0,1\}^* \times \{0,1\}^*$ *be an NP relation, where* x *denotes a statement and* w *denotes a corresponding witness. A zero-knowledge argument of knowledge for* R *is an interactive protocol* $(\mathcal{P}, \mathcal{V})$ *between a prover* $\mathcal{P}$ *and a verifier* $\mathcal{V}$ *satisfying the following properties:*

- ***Completeness.*** *For any* $(x, w) \in \mathsf{R}$, $\Pr[\mathcal{V}(x) = 1 \mid \mathcal{P}(x, w)] \geq 1 - \delta_c$.
- ***Soundness*** *There exists a probabilistic polynomial-time extractor* $\mathcal{E}$ *such that for any statement* x *and any probabilistic polynomial-time cheating prover* $\hat{\mathcal{P}}$*, if* $\Pr[\mathcal{V}(x) = 1 \mid \hat{\mathcal{P}}(x)] > \delta_s + \varepsilon$ *for some non-negligible* ε*, then* $\mathcal{E}$ *can extract in polynomial time a witness* w *such that* $(x, w) \in \mathsf{R}$ *via accessing* $\hat{\mathcal{P}}$ *in a black-box manner.*
- ***(Honest-Verifier) Zero-Knowledge.*** *There exists a probabilistic polynomial-time simulator* $\mathcal{S}$ *such that, for any* $(x, w) \in \mathsf{R}$*, the distribution of the view of an honest verifier* $\mathcal{V}$ *interacting with* $\mathcal{P}(x, w)$ *is computationally indistinguishable from the output distribution of* $\mathcal{S}(x)$*.*

Here, δ_c *and* δ_s *denote the completeness and soundness errors, respectively. The soundness property is required to hold only against p.p.t provers.*